

Proof. To prove the correctness of the algorithm, we need to just prove the correctness of the function *removeCyclicRelation* as it is the contribution of my thesis. The rest of the algorithm is an implementation of building the 2-SAT equivalence classes [3].

we consider we have a connected graph (we can solve a disconnected graph by considering the connected component and making sure that the connected comps drawings don't overlap horizontally). The set of solutions of the 2-SAT formula contains false positives, thus the goal is to restrict the set of results to not include these false positive. Therefore adding these restrictions won't lead to not respecting the planarity clause.

Bruekner et al. [1] shows how to build a hanani-tutte drawing Γ^* from a truth assignement of the 2SAT formula of the graph G , this transformation trades planarity for transitivity, thus we can focus on the planarity by transforming the hanani-tutte drawing to a planar drawing based on Fulek et al [2]. From a strong hanani-tutte we can get to a weak hanani-tutte if we can get rid of all the adjacent edges that crosses oddly [2]. One case of odd crossing is that the induced hanani-tutte have at least $e_1 = (v_0, v_1), e_2 = (v_0, v_2), e_3 = (v_0, v_3) \in E(G^*)$ with e_1 crosses oddly e_2 and e_3 lay in between e_1 and e_2 in the rotation of v_0 and without losing generality e_1 is above e_2 in the rotation of v_0 . This can happen in the hanani-tutte drawing induced by an assignment of the 2-SAT formula only when we have a cyclic relation between adjacent edges of the original graph G . Because in the induced hanani-tutte G^* we can have this case when v_3 forces the edges e_1 and e_2 to switch their order and v_3 is the closest vertex to the shared endpoint v_0 which means the new reversed order stays in the rotation in the shared vertex v_0 of the adjacent edges e_1, e_2, e_3 , which can only happen if there is a cyclic relation between some distinct endpoints from the same level of some adjacent edges. To solve this we can unify the equivalent classes that contain the relative order of the adjacent edges. This won't affect the planarity clause, since we are just restricting the number of possible solutions from the 2-SAT formula. The second case is if there is no e_3 between e_1 and e_2 in the rotation of v_0 . We just need to cross e_1 and e_2 near v_0 , but we won't work on this case directly as it can be solved while at the same time solving the special case in the weak hanani-tutte drawing.

Getting rid of odd crossings of the adjacent edges will give us a weak hanani-tutte drawing, it is equivalent to x-monotone drawing beside one case that requires changing wether an edge e passes below or above a vertex v of the weak hanani-tutte which leads to a change in the 2-SAT formulation between the vertex v and some of the vertices from the same level of the original graph G . Fulek et al [2] shows us how to draw x-monotone drawing, starting with one vertex from the first level of G^* and adding one vertex at a time to our drawing. The special case is if our new vertex v_1 that we want to add doesn't have an edge with the last added vertex v_2 and we have at least two edges (e, f) that are going to v_1 with e above f in the rotation of v_1 and e passes below x_2 and f passes above x_2 and $G^* - x_1$ separates into multiple components. We can only get this case due to a cyclic relation of vertex v on a level l that has no incoming edges from the level $l - 1$ with some distinct endpoints in the level l and are part of adjacent edges from a source vertex v_0 in level $l - 1$. To not get to this case we can define same relative order between the distinct endpoints of the adjacent edges and the vertex that has no incoming edge in that level and that's by putting all these relative orders in the same equivalent class if we have

multiple ones.

We fill the warnings list with vertices with no incoming edges from the level before and the adjSource map contains the adjacent edges that are incoming from the level below. We give the same relative order of each vertex to the warning one for each group of adjacent edges. We also use the adjSource map to link all the relative orders of distinct endpoints of adjacent edges.

the synchronization turns to an equivalence class in LinkSynchronizations, I am not a contributor to this part. $\square$

```

1: function COMPUTE2SAT(emb)
2:   sync  $\leftarrow$  Map{}
3:   for all  $(i, \text{nodes}) \in \text{enumerate}(\text{emb})$  do
4:     Synchronize(sync)
5:   end for
6:   return LinkSynchronizations(sync)
7: end function

```

```

1: function FILLSYNC(sync,  $a, b, c, d$ )
2:   sync[( $a, b$ )].add( $c, d$ )
3:   sync[( $b, a$ )].add( $d, c$ )
4:   sync[( $c, d$ )].add( $a, b$ )
5:   sync[( $d, c$ )].add( $b, a$ )
6: end function

```

References

- [1] Guido Brückner, Ignaz Rutter, and Peter Stumpf. Level-planarity: Transitivity vs. even crossings. *The Electronic Journal of Combinatorics*, pages P4–22, 2022.
- [2] Radoslav Fulek, Michael J Pelsmajer, Marcus Schaefer, and Daniel Štefankovič. Hanani–tutte, monotone drawings, and level-planarity. *Thirty essays on geometric graph theory*, pages 263–287, 2013.
- [3] Bert Randerath, Ewald Speckenmeyer, Endre Boros, Peter Hammer, Alex Kogan, Kazuhisa Makino, Bruno Simeone, and Ondrej Cepek. A satisfiability formulation of problems on level graphs. *Electronic Notes in Discrete Mathematics*, 9:269–277, 2001.

```

1: function SYNCHRONIZE(sync)
2:   warnings  $\leftarrow$  []
3:   E  $\leftarrow$  []
4:   adjSource  $\leftarrow$  Map{}
5:   for all  $n \in \text{nodes}$  do
6:     hasIncomingEdge  $\leftarrow$  False
7:     for all adj  $\in n.\text{adjEntries}$  do
8:       if adj.isSource() then
9:         hasIncomingEdge  $\leftarrow$  True
10:        E.add(adj.theEdge())
11:        adjSource[adj.theEdge().source()].add( $n$ )
12:      end if
13:    end for
14:    if  $\neg$  hasIncomingEdge then
15:      warnings.add( $n$ )
16:    end if
17:  end for
18:  for all  $(f, s) \in \text{combinations}(E, 2)$  do
19:     $a \leftarrow f.\text{source}()$ 
20:     $b \leftarrow f.\text{target}()$ 
21:     $c \leftarrow s.\text{source}()$ 
22:     $d \leftarrow s.\text{target}()$ 
23:    if  $a = b$  or  $c = d$  then
24:      continue
25:    end if
26:    fillSync(sync, a, b, c, d)
27:  end for
28:  removeCyclicRelations(sync, warning, adjSource);
29:  return sync
30: end function

```

```

1: function REMOVECYCLICRELATIONS(sync, warnings, adjSource)
2:   for all warning  $\in$  warnings do
3:     firstMapWarningSource  $\leftarrow$  True
4:     for all source  $\in$  keys(adjSource) do
5:       firstRelBetweenAdjacentEdgesFromSource  $\leftarrow$  True
6:       for all  $v \in$  adjSource[source] do
7:          $c \leftarrow v$ 
8:          $d \leftarrow$  warning
9:         if firstMapWarningSource then
10:           $a \leftarrow v$ 
11:           $b \leftarrow$  warning
12:          firstMapWarningSource  $\leftarrow$  False
13:        end if
14:        fillSync(sync, a, b, c, d)
15:      end for
16:      for all  $(v_1, v_2) \in \text{combinations}(\text{adjSource}[\text{source}], 2)$  do
17:        if  $v_1.\text{index}() < v_2.\text{index}()$  then
18:           $c \leftarrow v_1$ 
19:           $d \leftarrow v_2$ 
20:          if firstRelBetweenAdjacentEdgesFromSource then
21:             $a \leftarrow v_1$ 
22:             $b \leftarrow v_2$ 
23:            firstRelBetweenAdjacentEdgesFromSource  $\leftarrow$  False
24:          end if
25:          fillSync(sync, a, b, c, d)
26:        end if
27:      end for
28:    end for
29:  end for
30: end function

```

```

1: function LINKSYNCHRONIZATIONS(sync)
2:   eq  $\leftarrow$  HashMap{}
3:   cnt  $\leftarrow$  0
4:   for all  $(a, b) \in \text{sync}$  do
5:     if  $a.\text{index}() > b.\text{index}()$  then
6:       Swap  $a, b \leftarrow b, a$ 
7:     end if
8:     if  $(a, b) \in \text{eq}$  then
9:       continue
10:    end if
11:    eq[ $(a, b)$ ]  $\leftarrow$  Set{}
12:    eq[ $(b, a)$ ]  $\leftarrow$  Set{}
13:    todo  $\leftarrow [(a, b)]$ 
14:    while todo is not empty do
15:       $(c, d) \leftarrow \text{todo.pop}()$ 
16:      eq[ $(c, d)$ ]  $\leftarrow$  eq[ $(a, b)$ ]
17:      eq[ $(d, c)$ ]  $\leftarrow$  eq[ $(b, a)$ ]
18:      eq[ $(a, b)$ ].update(sync[ $(c, d)$ ])
19:      eq[ $(b, a)$ ].update(sync[ $(d, c)$ ])
20:      for all  $(e, f) \in \text{sync}[c, d]$  do
21:        if  $(e, f) \notin \text{eq}$  then
22:          todo.append( $((e, f))$ )
23:        end if
24:      end for
25:    end while
26:  end for
27:  return eq
28: end function

```
